



Parcours AI Engineer

SOUTENANCE PROJET 8

“Traiter les images pour le système embarqué de voitures autonomes”

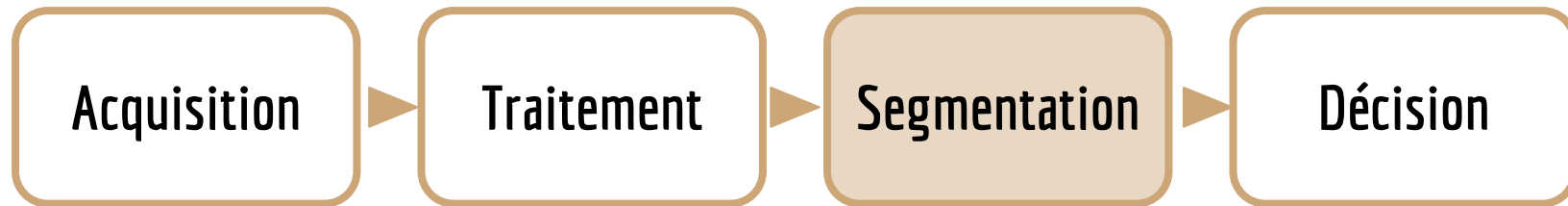
Stéphanie Duhem - Juillet 2026



00. CONTEXTE & OBJECTIFS

LE CONTEXTE

- **“Future Vision Transport”** : Entreprise spécialisée en systèmes embarqués de vision pour véhicules autonomes.
- **Mon rôle** : bloc segmentation d’images dans la pipeline R&D.



OBJECTIFS

- **1 modèle** : Entraîner pour identifier 8 catégories Cityscapes
- **1 API** : Déployer sur le Cloud pour les prédictions
- **1 Application Web** : Pour visualiser les résultats

CONTRAINTES TECHNIQUES

- **Rapidité** : Prédiction en temps réel pour le système embarqué ⚡
- **Qualité** : 8 classes dont des objets critiques (piétons) 🎯
- **Framework** : Keras imposé pour la cohérence de l'équipe 🛠️
- **Déploiement** : API & App web sur le Cloud ☁️
- **Data** : Classes minoritaires critiques (humains, objets) ⚠️

01-A. DATASET CITYSCAPES - VUE GLOBALE

2 975

images d'entraînement (notre
train + val)

500

images de validation (notre
test)

8

catégories sémantiques
agrégées

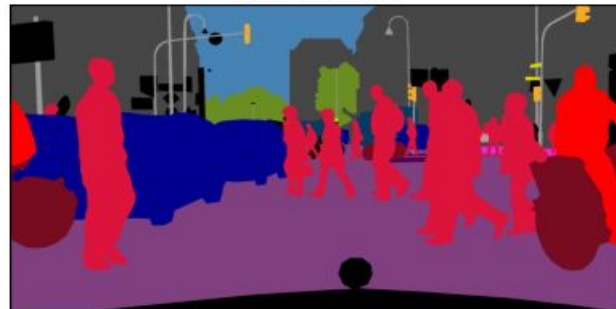
256×512

résolution utilisée

Image originale

Masque labels

Masque color



Contexte scénographique

- Allemagne, circulation citadine, prises depuis l'avant d'une Mercedes.
- Images diurnes, avec des météo variant de beau temps à couvert.

⇒ Axe d'enrichissement des données :

images nocturnes, autres villes, autre voiture, autres paysages (campagne, autoroute, pays), météo qui varie plus (brouillard, pluie etc..)

01-B. L'ENJEU CENTRAL DES CLASSES DÉSÉQUILIBRÉES

4

PROBLÈME

human + object = 3 % des pixels => modèle naïf les ignore



SOLUTION 1

- **Pondération des 8 classes** (Inverse Frequency), dès le batch

SOLUTION 2

- **Loss combinée** Dice + Focal, pour focus sur les classes difficiles

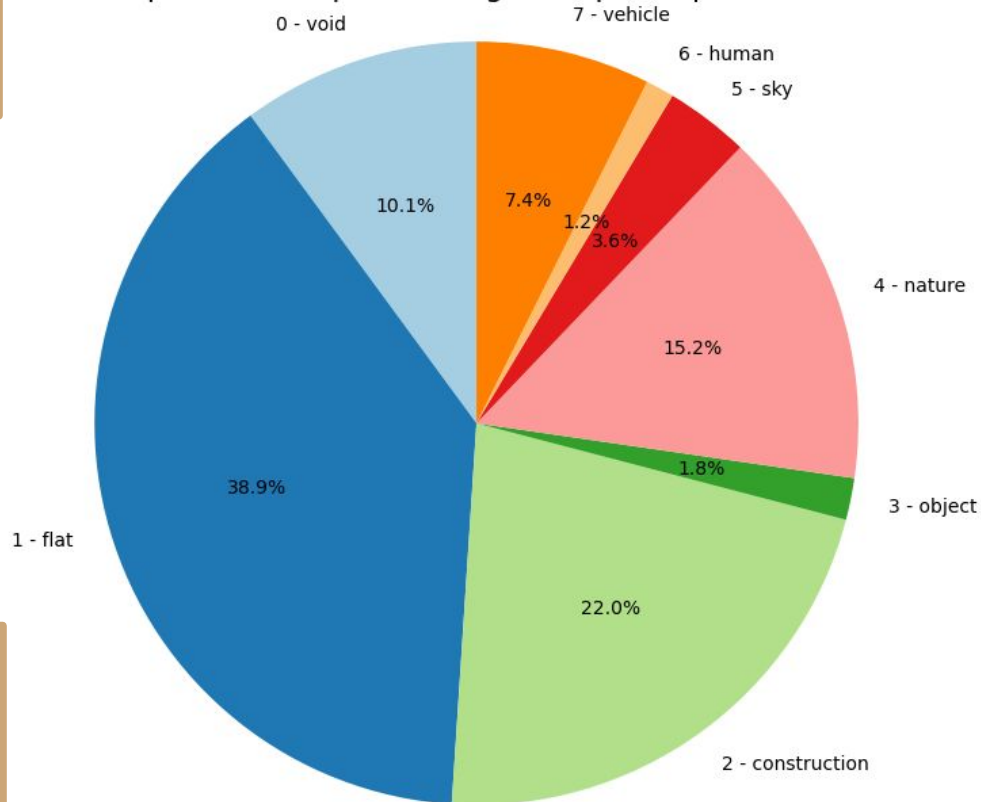


IMPACT

Meilleur rappel sur piétons et signalétique

Sécurité du modèle

Répartition en pourcentage des pixels par classe



02. PANORAMA DES APPROCHES DE SEGMENTATION

› Segmentation sémantique, d'instance et panoptique

Avant 2015

- Seuillage (Otsu 1979), clustering (K-means - Lloyd 1982), détection de contours (Canny 1986)

Deep Learning : CNN encodeur-décodeur & Backbones pré-entraînés - 2015-2020

- U-Net (Skip connections), FPN, DeepLabV3+
- Backbones pré-entraînés dès 2015 : Segnet avec VGG16, FPN avec Resnet
- DeepLabV3 (2017) sans décodeur => DeepLabV3+ (2018) avec décodeur

Optimisation pour déploiement embarqué - 2018-2021

- MobileNetV2 (2018), MobileNetV3 (2019), EfficientNet (2019)

Transformer & Foundation Models - 2021+

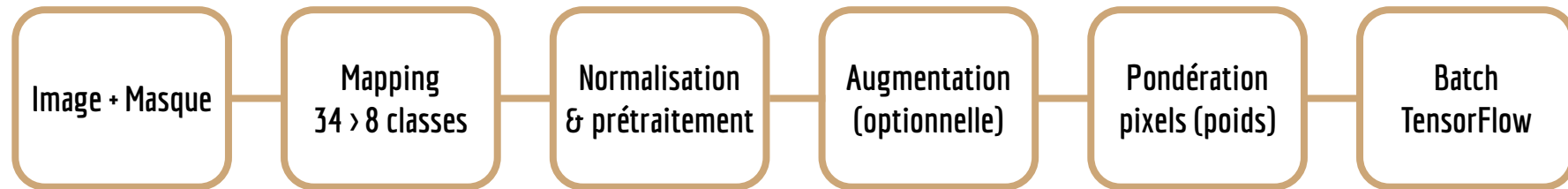
- ViT (2020) => SegFormer (2021), CLIP (2021), SAM (2023)
- Zero-shot learning (généralisation multi-tâches) avec CLIP ou SAM

Notre choix

Architectures U-Net avec backbones pré-entraînés
bon compromis précision / coût / contraintes embarquées

03. PIPELINE DE DONNÉES INDUSTRIABLE

La classe ImageSegmentationDataset



ImageSegmentationDataset — apports clés de ce générateur :

- **Réutilisable** : Héritage de la classe Keras PyDataset => compatible avec `model.fit()` et MLflow
- **Augmentations** : Alumentations intégré : flip horizontal, variations luminosité/contraste sur la paire image-masque
- **Poids** : Pondération par pixel dès le batch (Inverse Frequency), le triplet (image, masque, poids) intégré directement dans la loss
- **Multi-modèles** : Prétraitement spécifique à chaque backbone

04-A. BENCHMARK - ARCHITECTURES U-NET

Modèle	Paramètres	Taille	Entraînement	Usage dans le projet
UNet-mini	-0,19 M	-2 Mo	From scratch	BASELINE
MobileNetV3Small-UNet	-1,5-2 M	-17 Mo	Transfer Learning › Fine-tuning	Candidat naturel
VGG16-UNet	-15-20 M	-119 Mo	ImageNet	Challenger
ResNet50-UNet	-25 M	- 494 Mo	Transfer Learning › Fine-tuning	Upper bound

04-B. DEUX STRATÉGIES D'ENTRAÎNEMENT COMPARÉES

8

Transfer learning encodeur gelé

- **Encodeur (backbone)** : poids ImageNet gelés
- **Décodeur** : seul le décodeur U-Net est entraîné
- ✓ Convergence rapide
- ✓ Stable (BatchNorm stable)
- Représentations encodeur non adaptées

Fine-tuning complet Data Augmentation

- **Encodeur + Décodeur dégelés** : tout le réseau entraîné dès l'epoch 1
- **Enrichissement du dataset** par la data augmentation
- ✓ Adaptation complète à Cityscapes
- ✓ Potentiel de performance supérieur
- Plus long à converger
- Risque d'instabilité BatchNorm (ReduceLROnPlateau + early stopping aident à traverser cette phase)

04-C. CONTRÔLE DE L'APPRENTISSAGE & SUIVI EXPÉRIMENTAL

Keras callbacks

EarlyStopping

patience configurable
=> évite le sur-apprentissage

ReduceLROnPlateau

LR divisé si plateau
de val_loss détecté
=> convergence plus fine

ModelCheckpoint

Sauvegarde du meilleur
modèle (val_dice)
=> reproductibilité garantie

Suivi expérimental MLflow

- **Métriques loggées** : Dice, mIoU, Pixel Accuracy par epoch (train + val)
- **Paramètres** : Architecture, LR, batch size, freeze_encoder, augmentations
- **Artefacts** : Meilleur modèle .keras + courbes d'apprentissage

Chaque run est traçable et comparable.

La reproductibilité des résultats est garantie.

Reproductibilité

- Seeds fixées (PYTHONHASHSEED, numpy, TensorFlow) + assertion

05-A. COMPARAISON QUANTITATIVE

Architecture	Params	mIoU val	Dice val	IoU human	IoU vehicle	Taille	Inférence/img
U-Net mini	-0,2 M	0.349	0.430	0.011	0.202	2 Mo	0,18 sec
MobileNetV3 (gelé)	-1,8 M	0.601	0.639	0.328	0.690	9,7 Mo	0,29 sec
VGG16 (gelé)	-15 M	0.562	0.604	0.385	0.000 ⚠	119 Mo	0,21 sec
ResNet50 (gelé)	-25 M+	0.742	0.768	0.610	0.841	314 Mo	0,39 sec
MobileNetV3 (fine-t.) ✓	-1,8 M	0.661	0.696	0.418	0.749	17 Mo	0,32 sec
ResNet50 (fine-t.) 🌟	-25 M+	0.762	0.789	0.656	0.855	494 Mo	0,38 sec
MobileNetV3 (fine-t. + augm.)	-1,8 M	<i>0.652</i>	<i>0.691</i>	<i>0.385</i>	<i>0.747</i>	17 Mo	0,30 sec
ResNet50 (fine-t. + augm.)	-25 M+	<i>0.760</i>	<i>0.787</i>	<i>0.639</i>	<i>0.854</i>	494 Mo	0,38 sec

📌 Repère Dice — Métrique de qualité globale. Cityscapes : > 0,60 correct · > 0,75 bon

05-B. ANALYSE QUALITATIVE

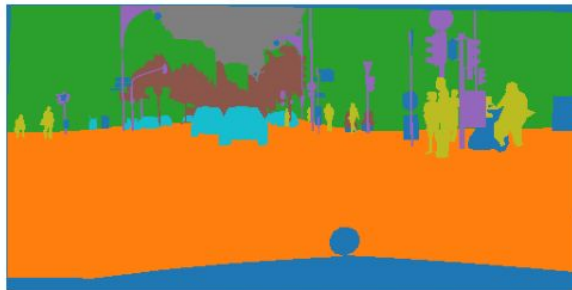
MobileNetV3

fine-tuned 50 epochs - 0,32 sec

Original Image



Ground Truth Mask



Predicted Mask



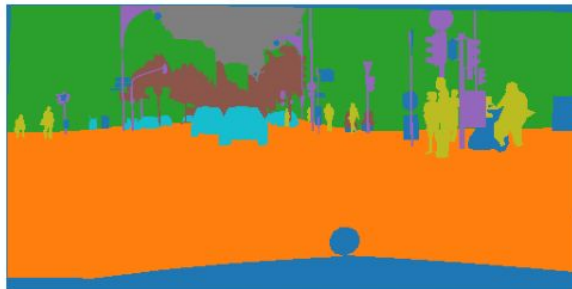
ResNet50

fine-tuned 50 epochs - 0,38 sec

Original Image



Ground Truth Mask

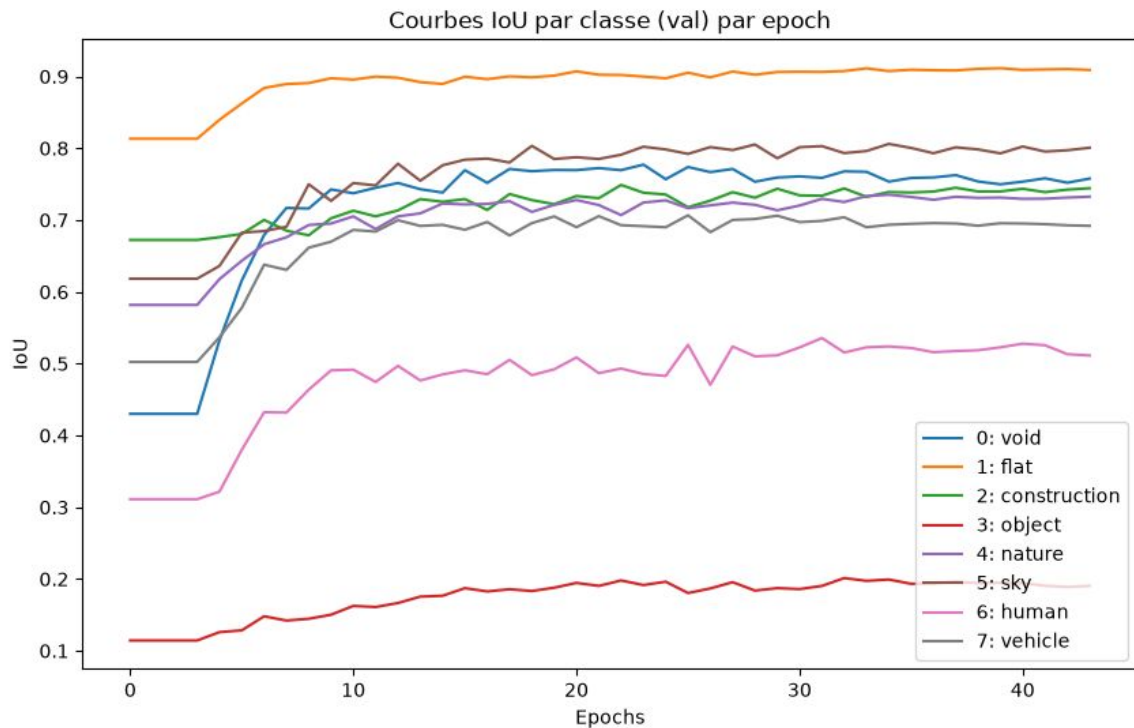
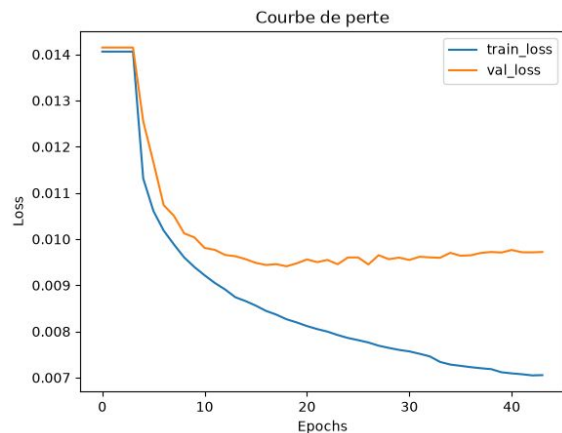
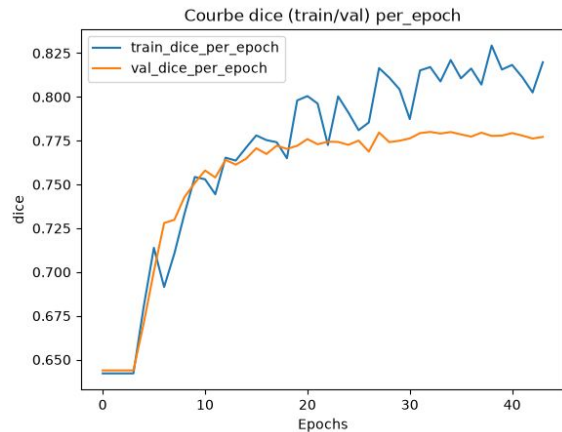


Predicted Mask

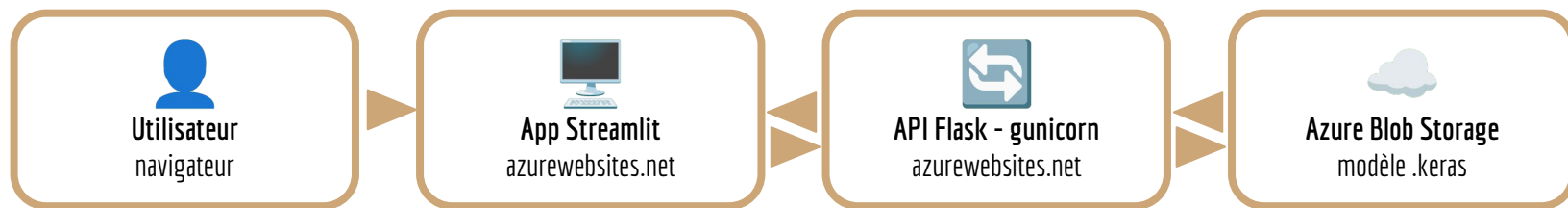


05-C. MODÈLE RETENU - MobileNetV3 fine-tuned

12



06-A. ARCHITECTURE DE DÉPLOIEMENT – AZURE CLOUD



Conteneurisation

- Docker (python:3.10-slim) — build via Azure Container Registry

Plan Azure

- App Service B1 (1 vCPU, 1.75 GB RAM, -13\$/mois)

Lazy loading

- Modèle chargé à la première requête et conservé en mémoire pour toutes les suivantes

/health :

- Endpoint de santé => démarrage instantané, pas de timeout Azure

API_URL - le découplage front/API

- Variable d'environnement => frontend découplé du backend

06-B. APPLICATION DE PRÉSENTATION DES RÉSULTATS

FONCTIONNALITÉS

- **Liste des images**
Affichage du nom des 11 images de test disponibles (chargées depuis Azure Blob Storage via `/preload`, mis en cache)
- **Sélection**
Choix d'une image dans le menu > appel à l'API `/select_img` > affichage côte à côte de l'image originale et du masque réel
- **Prédiction**
Bouton "PREDICT" > appel à l'API `/predict` > inférence du modèle MobileNetV3Small-UNet sur l'image sélectionnée
Ajout d'un **Spinner**
- **Visualisation**
Retour du masque prédit en base64, accompagné du nom du modèle, avec son score de Dice et de mIoU

Segmentation sémantique embarquée — Véhicule autonome

Dataset : Cityscapes (8 classes agrégées) · Modèle : MobileNetV3Small-UNet (fine-tuning)

Select a image

frankfurt_000000_000576_leftimg8bit.png

Original Image



Ground Truth Mask



\$ PREDICT \$

Modèle : MobileNetV3Small-UNet · val Dice = 0.696 · val mIoU = 0.661

Predicted Mask



06-B. APPLICATION DE PRÉSENTATION DES RÉSULTATS

SUITE

Segmentation sémantique embarquée — Véhicule autonome

Dataset : Cityscapes (8 classes agrégées) · Modèle : MobileNetV3Small-UNet (fine-tuning)

Select a image

frankfurt_000000_000576_leftImg8bit.png

frankfurt_000000_000576_leftImg8bit.png

frankfurt_000000_001016_leftImg8bit.png

frankfurt_000000_001751_leftImg8bit.png

frankfurt_000000_004617_leftImg8bit.png

frankfurt_000000_006589_leftImg8bit.png

frankfurt_000000_007365_leftImg8bit.png

frankfurt_000000_009969_leftImg8bit.png

frankfurt_000000_011074_leftImg8bit.png

Segmentation sémantique embarquée — Véhicule autonome

Dataset : Cityscapes (8 classes agrégées) · Modèle : MobileNetV3Small-UNet (fine-tuning)

Select a image

frankfurt_000000_000576_leftImg8bit.png

Original Image



Ground Truth Mask



\$ PREDICT \$

○ Prédiction en cours...

07-A. BILAN DU PROJET

RÉALISATIONS

- Pipeline ML complet et reproductible (seeds fixées)
- Classe **ImageSegmentationDataset** : un générateur de données Keras (industrialisable)
- 4 architectures comparées avec suivi **MLflow**
- **API Flask déployée sur Azure** (endpoint /predict)
- **Application Streamlit** déployée et fonctionnelle
- Code versionné sur **GitHub** (dev / main) avec 3 dépôts séparés P8_Segmentation_Images, P8_api_app, P8_front_app
- **Note technique** de 10 pages

LIMITES IDENTIFIÉES

- **Embarqué**
Le meilleur modèle, ResNet50 (314 Mo), est inadapté à un déploiement matériel strict
- **Démo**
MobileNetV3Small (~17 Mo) déployé, un compromis vitesse/performance sur CPU B1
- **Données**
Cityscapes offre peu de variété
- **Runs**
Comparaisons limitées par les ressources de calcul

07-B. PERSPECTIVES D'AMÉLIORATION

Données & robustesse

- **Dataset** Foggy / Rain Cityscapes pour conditions dégradées
- **Stratégie de pondération** : Test Median Frequency Balancing vs Inverse Frequency
- **Augmentations** géométriques plus agressives

Modèle & performance

- **Dégel** progressif de l'encodeur (layer-wise unfreezing)
- **Conversion du format du** modèle en TFLite avec quantification INT8 pour réduire taille et latence
- **Automatisation** du déploiement via pipeline CI/CD (GitHub Actions)

Applicatif & système

- **Segmentation** d'instance ou panoptique (compter piétons)
- Connexion à un module de décision prototype
- **Diminution du temps d'inférence** avec GPU dédié ou ONNX Runtime pour inférence temps réel



Merci de votre attention

